

UAPV/CERI - BASES DE LA PROGRAMMATION L1 S1 INFORMATIQUE

Feuille de TD n° 1

Dans cette UE nous programmons en python.
Pour afficher « Bonjour » on écrit : `print("Bonjour")`
Pour afficher le contenu d'une variable A, on écrit : `print(A)`;
On peut aussi faire `print("La valeur contenue dans la variable A est », A )`
Pour lire, on utilisera l'instruction : `a=input("Entrez une valeur: »)`

EXERCICE 1

Écrire un programme qui demande à l'utilisateur 2 entiers qu'il met dans deux variables a et b et qui affiche « plus grand » si a est supérieur à b, sinon il affiche « plus petit ou égal».
Modifier le programme pour qu'il affiche : « plus grand », « plus petit», ou « égal».

EXERCICE 2

Écrire un programme qui demande à l'utilisateur une longueur L et une largeur l (la longueur doit être supérieure ou égale à la largeur et $L \geq l$) et qui affiche la surface du rectangle correspondant. Si la condition sur les longueurs n'est pas respectée le programme affiche le message : « Données incohérentes »

EXERCICE 3

Écrire un programme qui lit 6 entiers et qui les mets dans 6 variables : a, b, c, d, e, f. Ensuite, si au moins 3 variables sont non nulles le programme doit afficher « la saisie est correcte », sinon il affiche « la saisie est incorrecte ».

EXERCICE 4

Écrire un programme qui lit 3 nombres qu'il met dans trois variables a, b et c et qui les réordonne de façon à ce que le contenu de a soit inférieur ou égal à celui de b et celui de b soit inférieur ou égal à celui de c. Le programme affiche a, b et c pour vérification.

EXERCICE 5

Écrire un programme qui permet de saisir 10 valeurs et d'affiche la plus petite valeur, la plus grande valeur et la moyenne. Ce programme ne doit utiliser que 4 variables.

EXERCICE 6

Écrire un programme qui saisit les coordonnées (entiers) de deux points et qui affiche la distance (double) les séparant.

EXERCICE 7

Écrire un programme qui saisit les coordonnées (entiers) de 3 points et qui dit si les trois points forment un triangle rectangle.

EXERCICE 8

Écrire un programme qui demande à l'utilisateur une opération (*,+,-,/), suivie de deux opérandes. Le programme doit afficher ensuite le résultat de l'opération appliquées aux deux opérandes. Les opérandes et les résultats doivent être des entiers.

Attention : pensez à interdire la division par 0.0

Exécution :

Entrez l'opération : *

Entrez la première opérande : 3 Entrez la deuxième opérande : 4 Le résultat de 3*4 est 12

EXERCICE 9

Écrire un programme qui demande 3 valeurs a ,b et c de type double et affiche les solutions de l'équation $ax^2+bx+c=0$. On vérifiera que a est différent de 0.

EXERCICE 10

Écrire un programme qui reçoit un nombre à 4 chiffres et qui affiche les chiffres un à un avec un espace les séparant. Le programme doit vérifier d'abord que le nombre contient bien 4 chiffres. Dans le cas contraire le programme affiche un message : « le nombre ne contient pas 4 chiffres »

BASES DE LA PROGRAMMATION L1 S1 INFORMATIQUE

Feuille de TD n° 2

Les boucles

EXERCICE 1

Écrire un programme qui demande à l'utilisateur de lui fournir un nombre entier positif et inférieur à 100 et ceci jusqu'à ce que la réponse soit satisfaisante. Le dialogue avec l'utilisateur se présentera ainsi: donnez un entier positif inférieur à 100 : 453

donnez un entier positif inférieur à 100 : 25

merci pour le nombre 25

EXERCICE 2

Écrire un programme qui demande à l'utilisateur un nombre entre 2 et 9 et affiche la table de multiplication de celui-ci. La saisie du nombre est bloquante : tant que le nombre n'est pas compris entre 2 et 9 le programme redemande...

EXERCICE 3

A- Écrire un programme qui saisit une suite de valeurs entières non nulles. A la fin de la saisie, le programme doit afficher la moyenne des valeurs saisies. La saisie s'arrête quand l'utilisateur donne la valeur 0.

B- Écrire un programme qui saisit une suite de valeurs entières non nulles. A la fin de la saisie, le programme doit afficher la moyenne des valeurs saisies, la valeur minimale et la valeur maximale. Les trois valeurs doivent être calculées en utilisant une seule boucle. La saisie s'arrête quand l'utilisateur donne la valeur 0.

EXERCICE 4

Écrire un programme qui saisit une suite de valeurs entières non nulles. A la fin de la saisie, le programme doit afficher la moyenne des valeurs paires et la moyenne des valeurs impaires. La saisie s'arrête quand l'utilisateur donne la valeur 0. Attention la division par zéro est une opération interdite.

EXERCICE 5

A- Écrire un programme qui permet de déterminer le juste prix (juste prix = 42). L'utilisateur saisit un nombre entier. Le programme répond : « inférieur » si le prix indiqué est supérieur à 42, il répond :

« supérieur » si le prix indiqué est inférieur à 42 et il répond : « BRAVO! » s'il trouve le prix exact. Tant que le prix n'est pas le bon, l'utilisateur doit retenter sa chance en saisissant un nouveau nombre.

B- Modifiez le programme précédent afin que le juste prix soit généré aléatoirement et pas codé en dur à 42, qui sera évidemment inconnu de l'utilisateur.

En plus le programme doit, à la fin de chaque partie, afficher le le nombre de coups de la partie et proposer à l'utilisateur s'il veut continuer le jeu. Si la réponse est « o » alors le programme lance une nouvelle partie, sinon le programme affiche le nombre de coups moyen global et affiche « A bientôt » avant d'arrêter.

EXERCICE 6

Écrire un programme qui permet de dessiner un rectangle d'étoiles *. Le programme a besoin de 2 entrées : la largeur et la hauteur du rectangle (que l'utilisateur doit fournir).

EXERCICE 7

Écrire un programme qui permet de dessiner X avec des *. Le programme a besoin d'une entrée : la taille de la forme. La taille doit être un nombre entier impair compris entre 15 et 20 (inclus). Tant que le nombre saisi ne vérifie pas la condition, le programme redemande la valeur de la taille.

EXERCICE 8

Écrire un programme qui calcule et affiche la factorielle d'un entier $n > 0$: $n! = n*(n-1)*(n-2)*...*1$; par définition $0! = 1$.

EXERCICE 9

Un nombre est dit premier s'il n'est divisible que par 1 par et lui même. Écrire un programme qui saisit un entier est qui affiche « PREMIER » s'il est premier et « PAS PREMIER » sinon. Modifier le programme pour qu'il saisisse deux entiers positifs et qu'il affiche tous les entiers premiers entre les deux.

TD3: les fonctions

L1 Math-info, 2025-2026

Dans cette série nous utiliserons seulement while pour exprimer les boucles

1- Définissez une fonction `surfCercle(R)`. Cette fonction doit renvoyer la surface (l'aire) d'un cercle dont on lui a fourni le rayon R en argument. Par exemple, l'exécution de l'instruction :

`print(surfCercle(2.5))` doit donner le résultat 19.635

2- Définissez une fonction `maximum(n1,n2,n3,n4)` qui renvoie le plus grand des 4 nombres n1, n2, n3 et n4 fournis en arguments. Par exemple, l'exécution de l'instruction :

`print(maximum(1,5,4,6))` doit donner

La valeur maximale est : 6.

3- Nous souhaitons résoudre l'équation : $ax^2+bx+c=0$ pour tous a, b et c.

Si $a == 0$

 si $b==0$

 si $c==0$

$S=R$

 else

 il n'y a pas de solution

 else

$S=\{-c/b\}$

else

$\Delta =$

 si $\Delta == 0$

$S = \{-b/2a\}$

 sinon si $\Delta > 0$

 sinon

3.1 Ecrire la fonction qui calcule le discriminant (Delta)

3.2 Ecrire la fonction, **resoudre**, qui résout l'équation. Attention la fonction ne doit faire aucun affichage.

3.3 Ecrire une fonction d'affichage qui permet après avoir utilisé la fonction résoudre, d'afficher le résultat de la résolution. Voici un exemple:

Entrez a, b et c :

0 2 1

Votre équation admet une seule solution : -0.5

4.1 Une action est cotée à 57€. Sa valeur augmente de 3% tous les mois. Écrire une fonction en Python qui renvoie le nombre de mois à attendre pour que sa valeur dépasse 200€.

4.2 Proposer une généralisation de la fonction précédente : étant donnée le prix initial d'une action, étant donné le taux d'augmentation de l'action à chaque mois (dans 4.1 le taux est 0.03), la fonction doit renvoyer le nombre de mois pour atteindre une valeur donnée.

5- Écrire une fonction « somme » qui prend en argument un entier positif et qui renvoie la somme de tous les chiffres le composant. Par exemple, « `print(somme(1234))` » affichera 10

6- Écrire une fonction Python bissextile prenant en entrée un entier représentant une année et retournant un booléen indiquant si l'année est bissextile. On rappelle qu'une année est bissextile si elle possède un 29 février. Une année est bissextile si elle est multiple de 4 mais pas multiple de 100, ou si elle est multiple de 400.

Ex : `bissextile(2024)` doit renvoyer True, `bissextile(1800)` doit renvoyer False et `bissextile(2000)` doit renvoyer True

7-

A- Écrire une fonction qui détermine si deux nombres réels sont identiques à un epsilon près. Exemple : 2.01 et 2.005 sont identiques à 0.01 près, car $|2.01 - 2.005| = 0.005 < 0.01$.

On considère que deux nombres réels sont identiques si ils le sont à $1e-6$.

B- Écrire une fonction qui prend en argument 3 nombres réels et qui retourne True/False selon qu'ils peuvent représenter les carrés des longueurs d'un triangle rectangle.

C- Écrire une fonction qui prend en argument 4 nombres réels et qui retourne True/False si trois d'entre eux peuvent représenter les carrés des longueurs d'un triangle rectangle.

8-

Voici la méthode de décomposition d'un entier en facteurs premiers:

Entrée : un entier naturel $n > 1$

Sortie : une chaîne de caractères qui représente la décomposition de n en facteurs premiers

1. Initialiser une chaîne `resultat` vide.
2. Tant que $n > 1$, faire :
 - a. Trouver le plus petit diviseur premier d de n .
 - b. Si `resultat` est vide, mettre d dans `resultat`.
 - c. Sinon, ajouter " x " puis d à la fin de `resultat`.
 - d. Remplacer n par n/d (division entière).
3. Quand $n = 1$, arrêter.

A- Écrire une fonction **est_premier(n)** qui retourne True si n est un nombre premier, sinon False.

B- Écrire une fonction **plus_petit_diviseur(n)** qui utilise `est_premier` et retourne le plus petit diviseur premier de n (supérieur à 1).

C- Écrire une fonction **decomposition_facteurs(n)** qui utilise plusieurs appels à `plus_petit_diviseur` pour produire la décomposition en facteurs premiers de n .

Ici exceptionnellement on va utiliser une variable de type **str** (chaîne). On début on fait : **resultat=""**. Imaginons que **resultat** vaut à un moment "2x2" et qu'on veut ajouter **d** (un entier) comme facteur, alors on fait simplement : **resultat=resultat+str(d)**

TD3 Liste de listes

Bases de la programmation

L1 informatique, mathématique et physique

2025-2026

1- Une matrice en python peut être implantée en utilisant une liste de listes. Chaque ligne de la matrice est une sous liste. Ecrire une fonction « Zeros » qui prend en argument deux entiers n et m et qui crée une matrice initialisée avec des 0. La matrice créée est une liste de n sous-listes. Chaque sous liste est une liste d'entiers contenant m entiers (initialisés à 0). Utiliser la fonction append en partant de listes vides

Exemple avec n=5 et m=4

```
[  
  [1, 2, 3, 4],  
  [4, 5, 6, 7],  
  [8, 9, 1, 4],  
  [1, 2, 5, 4],  
  [1, 2, 2, 4],  
]
```

2- Ecrire une fonction « aleatoire » qui crée une matrice d'une taille donnée (n et m) et qui l'initialise aléatoirement de valeurs entre deux entiers donnés.

A partir de maintenant, toute fonction prenant en argument une matrice, prend aussi en argument ses dimensions (n et m)

3- Ecrire une fonction d'affichage.

4- Ecrire une fonction qui renvoie le nombre de fois où une valeur donnée apparaît dans une matrice (le nombre d'occurrences d'une valeur)

5- Ecrire une fonction qui dit si deux lignes données d'une matrices donnée sont identiques

6- Ecrire une fonction qui dit si dans une matrice donnée ils existent deux lignes identiques. Vous devez utiliser la fonction de la question 5-

7- Ecrire une fonction qui dit si dans une matrice donnée il existent deux colonnes identiques (utiliser la même méthode qu'en 5 et 6)

8- Ecrire une fonction qui renvoie le produit de deux matrices (n'oubliez pas de renvoyer les nouvelles dimensions).

9- Ecrire une fonction qui dit si une matrice est symétrique

10- Une liste L est dite triée si et seulement si pour tout $i < n-1$ (n étant la taille de la liste), $L[i] < L[i+1]$.

Ecrire une fonction qui prend en argument une liste et qui renvoie `True` si la liste est triée et `False` sinon.

Une matrice est dite triée si et seulement si toutes les sous-listes (les lignes) sont triées

11-

Écrire une fonction qui prend en arguments une liste L et un entier n (avec n inférieur à la taille de la liste) et qui pour tout $i < n - 1$, si $L[i] > L[i+1]$, on échange leurs valeurs.

- S'il n'existe aucun i tel que $L[i] > L[i+1]$, la fonction renvoie `True`.
- Sinon, elle renvoie `False`.

Utiliser cette fonction pour trier une liste.

12- Ecrire une fonction qui permet de trier une matrice.